

1 Polecenie

Indeks 353548 $\equiv_4 0 \Rightarrow$ **Podpunkt 0:** W OCTAVE lub MATLAB napisać funkcję obliczającą wartość dystrybuanty rozkładu $\chi^2(k)$ w punkcie x . Dana jest gęstość rozkładu $\chi^2(k)$:

$$f(x) = \frac{1}{2^{\frac{k}{2}} \Gamma(\frac{k}{2})} x^{\frac{k}{2}-1} e^{-\frac{x}{2}}, \quad k \in \mathbb{N}, x \in [0, \infty)$$

2 Rozwiązanie

2.1 Dystrybuanta

Zaczynamy od policzenia dystrybuanty.

$$F(x) = P(X \leq x) = \int_0^x f(t) dt = \int_0^x \frac{1}{2^{\frac{k}{2}} \Gamma(\frac{k}{2})} t^{\frac{k}{2}-1} e^{-\frac{t}{2}} dt$$

Wprowadzamy podstawienie $u = \frac{t}{2} \Rightarrow t = 2u, dt = 2du$ i wyciągamy stałe.

$$\int_0^x \frac{1}{2^{\frac{k}{2}} \Gamma(\frac{k}{2})} t^{\frac{k}{2}-1} e^{-\frac{t}{2}} dt = \int_0^{\frac{x}{2}} 2 \cdot \frac{1}{2^{\frac{k}{2}} \Gamma(\frac{k}{2})} (2u)^{\frac{k}{2}-1} e^{-u} du = \frac{1}{2^{\frac{k}{2}} \Gamma(\frac{k}{2})} \cdot 2^{\frac{k}{2}-1} \cdot 2 \cdot \int_0^{\frac{x}{2}} u^{\frac{k}{2}-1} e^{-u} du = \quad (1)$$

$$= \frac{1}{2^{\frac{k}{2}} \Gamma(\frac{k}{2})} \cdot 2^{\frac{k}{2}} \cdot \int_0^{\frac{x}{2}} u^{\frac{k}{2}-1} e^{-u} du = \frac{1}{\Gamma(\frac{k}{2})} \int_0^{\frac{x}{2}} u^{\frac{k}{2}-1} e^{-u} du \quad (2)$$

Otrzymujemy wzór na dystrybuentę:

$$F(x) = \frac{1}{\Gamma(\frac{k}{2})} \int_0^{\frac{x}{2}} u^{\frac{k}{2}-1} e^{-u} du = \frac{\gamma(\frac{k}{2}, \frac{x}{2})}{\Gamma(\frac{k}{2})},$$

gdzie γ oznacza niekompletną funkcję gamma.

2.2 Implementacja

Do obliczenia dystrybuanty potrzebujemy obliczyć dwie całki: `gamma_inc` i `gamma_fun`.

```
function res = z2(x, k)
    res = gamma_inc(k/2, x/2) / gamma_fun(k/2)
end
```

2.3 Metoda Romberga

Do liczenia całek przydatna może być metoda Romberga. Tworzymy *tablicę Romberga* - pierwszą kolumnę zapełniamy za pomocą złożonego wzoru trapezów, w każdym wierszu dzieląc pole pod wykresem na coraz więcej przedziałów. Wzór trapezów:

$$\int_a^b f(x) dx \approx \frac{b-a}{n} \left(\frac{f(a)}{2} + f(x_1) + \dots + f(x_{n-1}) + \frac{f(b)}{2} \right)$$

Znając wartości pierwszej kolumny kolejne kolumny liczymy rekurencyjnie dostając coraz lepsze przybliżenie całki.

```

function res = romberg(f, a, b)
    maxn = 15;
    R = zeros(maxn, maxn);

    R(1,1) = (b - a) * (f(a) + f(b)) / 2;

    for i = 2:maxn
        h = (b - a) / 2^(i-1);

        suma = 0;
        for k = 1:2^(i-2)
            suma += f(a + (2*k - 1)*h);
        end

        R(i,1) = 0.5 * R(i-1,1) + h * suma;

        for j = 2:i
            R(i,j) = R(i,j-1) + (R(i,j-1) - R(i-1,j-1))/(4^(j-1) - 1);
        end
    end

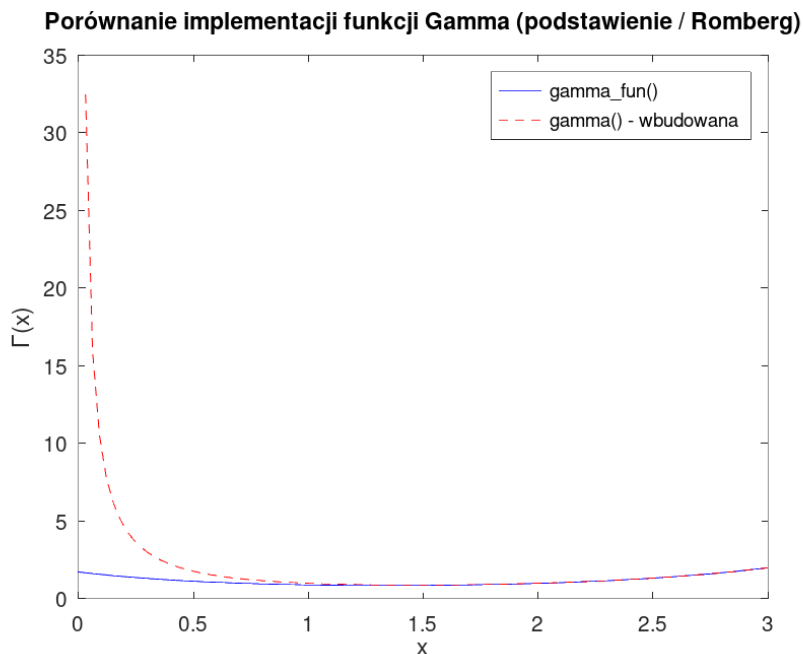
    res = R(maxn, maxn);
end

```

Używamy stałej $\text{maxn} = 15$, żeby wykonać 15 iteracji i dostać całkiem dobre przybliżenie wartości całki (ważne dla dużych x).

2.4 Funkcja gamma_fun

gamma_fun to całka na przedziale $[0; \infty)$. Mamy tu całkę niewłaściwą więc, którą problematycznie byłoby liczyć metodą Romberga - intuicyjnie nasuwa się chęć zmiany granic całkowania jakimś sprytnym podstawieniem np. $t = \frac{s}{1+s}$, żeby zamienić przedział $[0; \infty)$ na $[0; 1)$.



Widać z wykresu, że problem pojawia się dla małych wartości - implementacja funkcji Gamma za pomocą podstawienia i metody Romberga psuje się dla małych wartości (w okolicach jedynki i mniejszych). Lepszym rozwiązaniem w tym wypadku będzie użycie **przybliżenia Lanczosa**. Jest ono bardzo dokładne (na tyle, że jest stosowane jako domyślna implementacja funkcji Gamma w wielu bibliotekach tj. *GNU Scientific Library*, *Boost*, *CPython*) i pozwala na obliczenie funkcji Gamma z określoną precyzją, a sama implementacja i stałe potrzebne do obliczeń są łatwo dostępne.

Wzór Lanczosa dla funkcji Gamma wygląda następująco:

$$\Gamma(z) = \sqrt{2\pi} \left(z + g + \frac{1}{2}\right)^{z+\frac{1}{2}} e^{-(z+g+\frac{1}{2})} A_g(z),$$

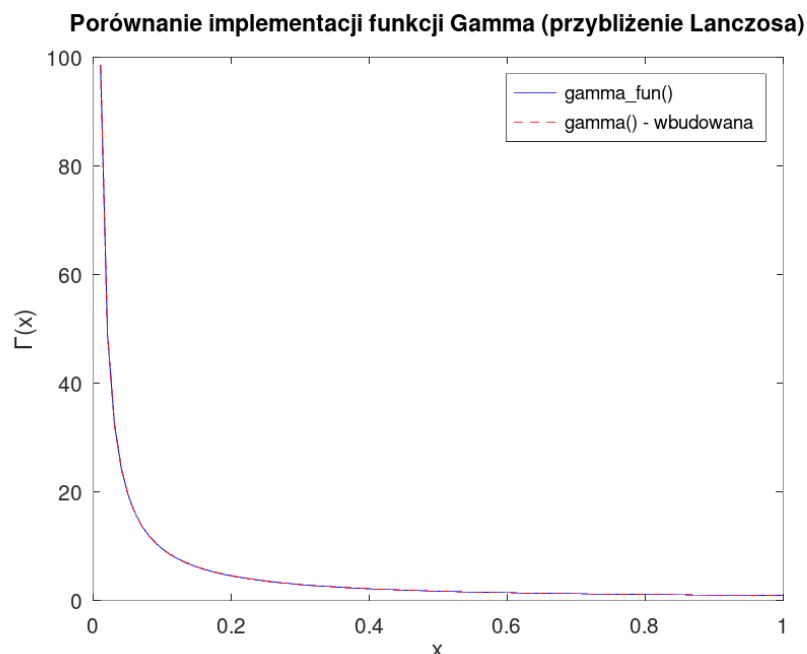
gdzie:

$$A_g(z) = \frac{1}{2}p_0(g) + p_1(g)\frac{z}{z+1} + p_2(g)\frac{z(z-1)}{(z+1)(z+2)} + \dots$$

W powyższych wzorach g jest stałą, a $p_k(g)$ zależnymi od niej współczynnikami (które ciężko policzyć, ale znane są ich wartości dla popularnych stałych). W mojej implementacji użyję stałej $g = 7$ z dziewięcioma współczynnikami ($n = 9$).

```
function res = gamma_fun(z)
% Współczynniki przybliżenia Lanczosa (g=7, n=9)
g = 7;
p = [
    ... (stałe)
];

if z < 0.5
    % gamma(z) * gamma(1-z) = pi/sin(pi * z)
    res = pi / (sin(pi * z) * gamma_fun(1 - z));
else
    z -= 1;
    x = p(1);
    for i = 2:9
        x += p(i) / (z + i - 1);
    end
    t = z + g + 0.5;
    res = sqrt(2 * pi) * t^(z + 0.5) * exp(-t) * x;
end
end
```



Nowa implementacja daje bardzo dokładne wyniki i pozwala uniknąć rozbieżnych wartości w okolicach jedynki.

2.5 Funkcja gamma_inc

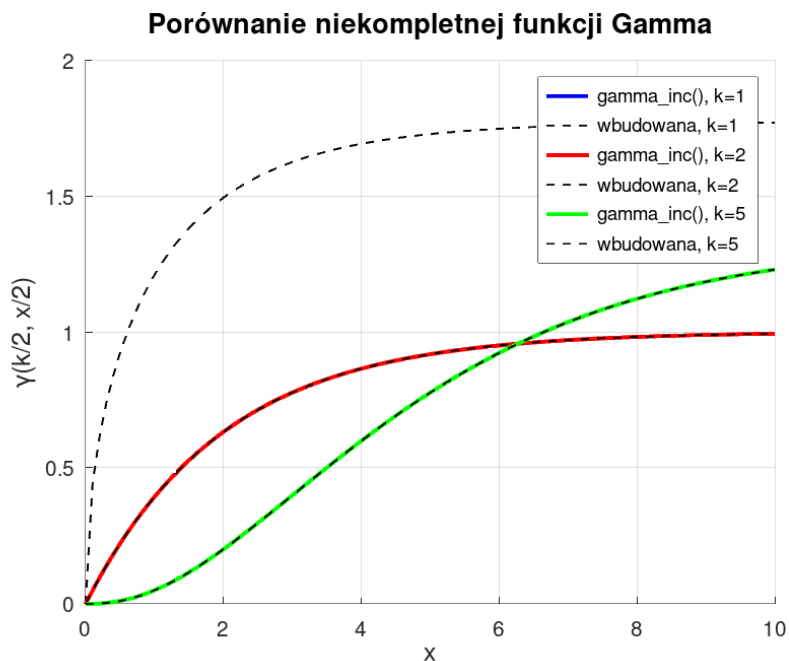
`gamma_inc` to funkcja pobierająca dwa argumenty: k , x . Granice całkowania są skończone, więc bez problemu można policzyć ją przy użyciu metody Rombera.

```

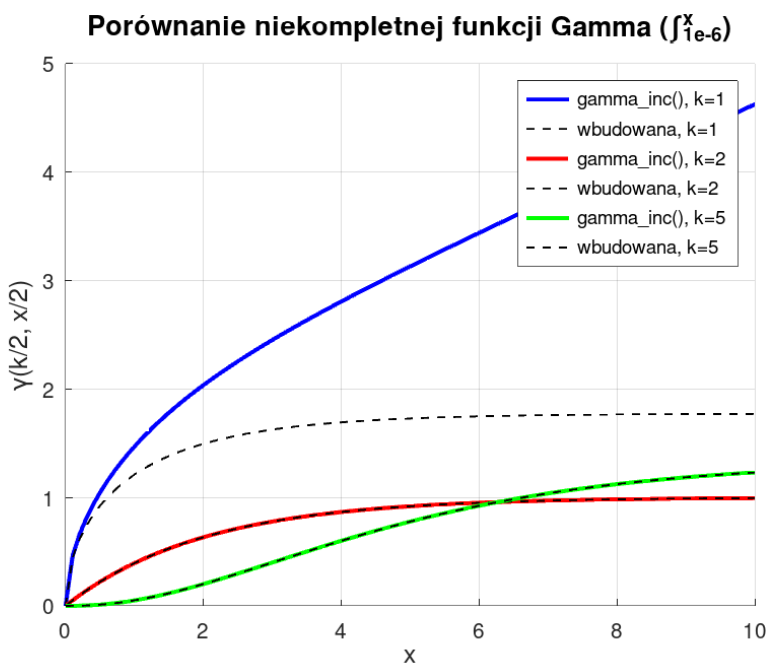
function res = gamma_inc(k, x)
    if x <= 0
        res = 0;
    else
        f = @(t) (t)^(k-1) * exp(-t);
        res = romberg(f, 0, x);
    end
end

```

Sprawdźmy działanie funkcji na wykresie dla wartości $k = 1, 2, 5$. W OCTAVE wbudowana funkcja `gamma` jest od razu normalizowana więc do testów używamy `y_builtin = gammainc(x/2, k/2) * gamma(k/2)`.



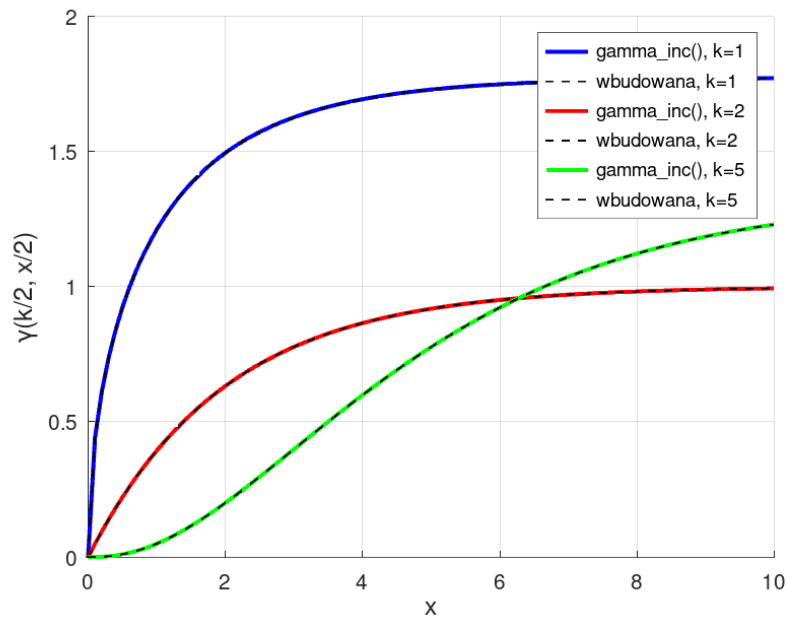
Widać, że obliczanie wartości całki metodą Romberga powoduje problem dla $k = 1$ (i również $k < 1$ - mimo, że $k \in \mathbb{N}$ można to poprawić, żeby konkurować z funkcją wbudowaną). Dzieje się tak dlatego, że romberg całkuje od 0, ale $f(0) = 0^{k-1} \cdot e^0$ - dla $k = 1$, czyli $k/2 = 0.5$ mamy $0^{-0.5}$ - dzielenie przez zero / Inf. Dla bardzo małych wartości k mogą też wystąpić duże rozbieżności.



Problem można rozwiązać podstawieniem $t = u^{\frac{1}{k}}$:

$$\gamma(k, x) = \int_0^x t^{k-1} e^{-t} dt = \frac{1}{k} \int_0^{x^k} e^{-u^{\frac{1}{k}}} du$$

Porównanie niekompletnej funkcji Gamma (podstawienie)



Ostatecznie:

```
function res = gamma_inc(k, x)
    if x <= 0
        res = 0;
    else
        g = @(t) exp(-(t^(1/k)));
        res = (1/k) * romberg(g, 0, x^k);
    end
end
```

3 Wyniki

Do testów używamy funkcji `chi2cdf` z pakietu `statistics`.

x	k	$z2$	<code>chi2cdf</code>	Różnica
0.0	2	0.00000000	0.00000000	0.00e+00
0.5	0.75	0.62641886	0.62641886	4.44e-16
1.0	2	0.39346934	0.39346934	6.11e-16
2.0	2	0.63212056	0.63212056	4.44e-16
5.0	2	0.91791500	0.91791500	2.33e-15
1.0	1	0.68268949	0.68268949	5.55e-16
1.0	4	0.09020401	0.09020401	2.04e-09
1.0	10	0.00017212	0.00017212	1.97e-10
3.0	5	0.30001431	0.30001416	1.51e-07

Nowa implementacja funkcji daje całkiem dobre wyniki, błędy numeryczne mogą pojawiać się dla większych liczb - funkcja Gamma bardzo szybko osiąga ogromne wartości. Zwykle nie przekroczymy granicy błędu $\epsilon = 0.0001$.